Original Scope Questions

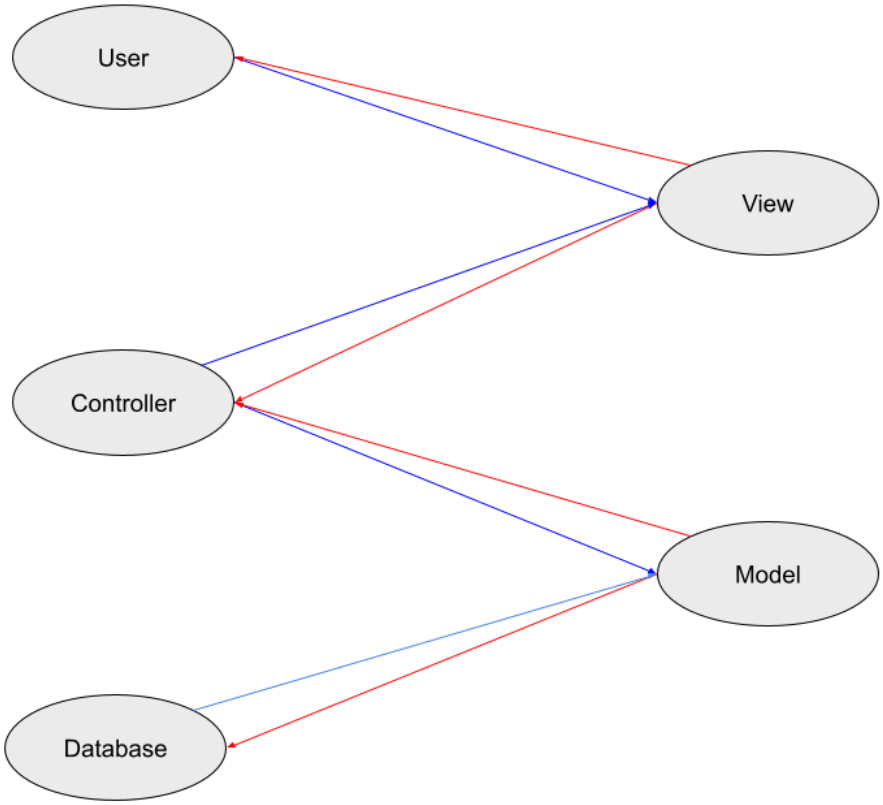
Project Description 1 Sentence: A project that allows the user to store all their passwords in a centralized location and to retrieve those passwords when they need them for the site.

Target Users or How will users use this: Users will download an executable that will run in the background allowing them to access it when in need of a password

User Actions

- Initial Setup
  - Initial Scope - Done
- Login
  - Initial Scope - Done
- Adding a Password
  - Initial Scope - Done
- Viewing Saved Passwords
  - Initial Scope - Done
- Delete a Password
  - Initial Scope - Done
- Logout
  - Initial Scope
- Check if account exists in DB already
  - Initial Scope - Done

System Diagram Rough Draft



1. User Interact With View (RED)
2. Controller knows that user interacted with View(RED)
3. Controller contacts Model for Data(RED)
4. Model Queries Database for Data(RED)
5. Database Sends Back Result Set(Blue)
6. Model Sends Data To Controller (Blue)
7. Controller Updates View(Blue)
8. View has been updated for User(Blue)

Desired Language and Tools

- Java For Backened
- JavaFX for front end
- MVC for Design
- Model is for data storage
- View is Java FX
- Controller is for modifying the data given from model
- H2 for Database

Specifics

<u>Goal is to be as specific as possible to how the operation works</u>

<u>We will then sudo code once we have an understanding of how it would work</u>

User<->View

- User interacts with the view

View<->Controller

- Controller is used to act on input from user and represent data based on what the user wants and asks the model

Controller<->Model

- Calls methods on the models to update data

Model<->Database

- We have to use and embedded database as we can not assume that the user has access to a database like MySQL

Classes

- Model
  - AccountInstance
  - AES
  - DBQueries
  - Session
  - SignUp
- View
  - AddAccountPopup
  - Create-account
  - First-scene
  - Login-view
  - Main-screen
- Controller
  - MainScreenController
  - ModelController
- Database Tables
  - Accounts

Additional Features

- Security Question for every password query
- Email verification
- Unique passwords
- Generated passwords
- Password strength tester
- Database table for suspicious logins
- Check for duplicate passwords

Problem Faced

Struggling to understand how AES would work to generate a key that would be the same every time for the user. Also would I have to ask the user every time for a specific piece of information any time they wanted to add a password.

Solution

I based the AES key off of the user's initial password to login into the app, this key would then be used to decrypt anything already in the database. Re-encrypted is what I initially thought to do but unfortunately that does not work as it can be easier for attackers to see the same cipher text and can assume it's the same string and work off of that. Instead we decrypt whatever information we need and compare it to user input. If it matches it is correct. We also created a standardized way for cipher text to be displayed in the database.

Encryption Standardized Format

7NjLPc/2QIiOdjY0YWEP+5OXCtd4OjtSw8u4ezyU3jrXHrjIGVDH - Example Cipher Text

8tsQKlCKyIHrQ3cYpxk/Fw== - Example Salt

In our application we have three things that make up the entire cipher text being the IV, Encrypted data, and GCM tag used for authentication. The IV is 12 bytes long, the Encrypted data is between the IV and GCM, and the GCM tag is 16 bytes long so for this example:

- The IV is: **7NjLPc/2QIiOdjY0YWEP**
- The encrypted data is: **+5OXCtd4OjtSw8u4ezyU3**
- The GCM tag is: **jrXHrjIGVDH**

The **IV** is a random character set essentially so that if the same plaintext gets encrypted the resulting cipher text is different which is why it is separated with a “+”.

The **GCM** first generates random bytes that get XOR with the plaintext. To produce new bit values that get encoded to different letters. It then also creates a checksum value as somewhat of a tamper seal that is 128 bits or 16 bytes long that if tampered with shows that the data is not authentic.

When we go to decrypt we use the IV and key we derived from logging in to then decrypt the just the cipher text as we know is between the IV and GCM. We only decrypt just to display and never store it in a long term variable/.

Class AccountInstance

java.lang.Object
com.example.passmanager.AccountInstance

Direct Known Subclasses:

SignUp

```
public class AccountInstance
extends Object
```

Represents a single stored account entry in the password manager.

Each instance holds the platform (service name), username, encrypted password, and the unique database ID for the account.

Field Summary

Fields

Modifier and Type	Field	Description
private int	**id**	
private String	**password**	
private String	**platform**	
private String	**username**	

Constructor Summary

Constructors

Constructor	Description
AccountInstance(int id, String password, String platform, String username)	Constructs an AccountInstance with the given account details.

Method Summary

All Methods Instance Methods Concrete Methods

Modifier and Type	Method	Description
int	**getId**()	Returns the database primary key ID associated with the account
String	**getPassword**()	Returns password associated with account, later gets encrypted and decrypted
String	**getPlatform**()	Returns the platform name for this account.
String	**getUsername**()	Returns the username for this account.
void	**setPassword**(String password)	Sets new password for the associated account
void	**setPlatform**(String platform)	Sets new platform name for this account.
void	**setUsername**(String username)	Sets new username for this account.

Methods inherited from class Object

clone , equals , finalize , getClass , hashCode , notify , notifyAll , toString , wait , wait , wait

Field Details

password

private String password

platform

```
private String  platform
```

username

```
private String  username
```

id

```
private int id
```

Constructor Details

AccountInstance

```
public AccountInstance(int id,
                       String  password,
                       String  platform,
                       String  username)
```

Constructs an AccountInstance with the given account details.

Parameters:

`id` - this is the primary key used in the H2 database

`password` - this password eventually gets encrypted when put into the database and decrypted when taken out

`platform` - the platform the user wants the associated username and password combo to be used for like GMAIL

`username` - the name for the account

Method Details

getId

```
public int getId()
```

Returns the database primary key ID associated with the account

Returns:

the account's ID

getPassword

```
public String  getPassword()
```

Returns password associated with account, later gets encrypted and decrypted

Returns:

the password

setPassword

```
public void setPassword(String  password)
```

Sets new password for the associated account

Parameters:

`password` - the new password to set

getPlatform

```
public String  getPlatform()
```

Returns the platform name for this account.

Returns:

the platform

setPlatform

```
public void setPlatform(String  platform)
```

Sets new platform name for this account.

Parameters:

`platform` - the platform name

getUsername

```
public String  getUsername()
```

Returns the username for this account.

Returns:

the username string

setUsername

```
public void setUsername(String  username)
```

Sets new username for this account.

Parameters:

`username` - the username

Class AddAccountController

java.lang.Object
com.example.passmanager.AddAccountController

```
public class AddAccountController
extends Object
```

JavaFX controller for AddAccountPupup.

Handles account creation withing the app with multiple validations occurring throughout

Field Summary

Fields

Modifier and Type	Field	Description
private javafx.scene.control.PasswordField	confirmPasswordField	
private javafx.scene.control.Label	errorLabel	
private javafx.scene.control.PasswordField	passwordField	
private javafx.scene.control.TextField	platformField	
(package private) DBQueries	query	
private javafx.scene.control.TextField	usernameField	

Constructor Summary

Constructors

Constructor	Description
AddAccountController()	

Method Summary

All Methods Instance Methods Concrete Methods

Modifier and Type	Method	Description
private void	**onAddClick**()	Handles - Text Field Inputs and validation, Password Requirements check, Creates a connection to database to call a query to add the account information in.
private void	**onCancelClick**()	When the pop-up screen to add an account appears there is a cancel button that closes this pop-up.
private String	**validatePassword**(String password)	Creates a standardized and secure way to accept passwords and write them into the database by iterating through the password and checking the length for the requirements

Methods inherited from class Object

clone , equals , finalize , getClass , hashCode , notify , notifyAll , toString , wait , wait , wait

Field Details

query

DBQueries query

usernameField

private javafx.scene.control.TextField usernameField

passwordField

```
private javafx.scene.control.PasswordField passwordField
```

confirmPasswordField

```
private javafx.scene.control.PasswordField confirmPasswordField
```

platformField

```
private javafx.scene.control.TextField platformField
```

errorLabel

```
private javafx.scene.control.Label errorLabel
```

Constructor Details

AddAccountController

```
public AddAccountController()
```

Method Details

validatePassword

```
private String  validatePassword(String  password)
```

Creates a standardized and secure way to accept passwords and write them into the database by iterating through the password and checking the length for the requirements

Parameters:

`password` - the plaintext password to derive a key from

Returns:

a String

onCancelClick

```
private void onCancelClick()
```

When the pop-up screen to add an account appears there is a cancel button that closes this pop-up. This specifically finds the username field in the pop-up window and closes it which is considered a Scene -> Window which gets cast as a Stage

onAddClick

```
private void onAddClick()
                throws Exception
```

Handles - Text Field Inputs and validation, Password Requirements check, Creates a connection to database to call a query to add the account information in. Query returns a result set in which we check if it's empty, if not we know there is an account with ID of 1. We then get the username of the previous query to find the salt with another query. With that salt and the session password we create the key to encrypt with and store it in the database and close the database.

Throws:

`Exception` - for SQL database connection and for encryption algorithm

Class AES

java.lang.Object
com.example.passmanager.AES

```
public class AES
extends Object
```

Class used to provide encryption dn decryption to the database and frontend. Creates a standardized way to store cipher text in the database so that we know how to decrypt it later when needed. Decryption only happens when data access is needed and is never stored anywhere.

Field Summary

Fields

Modifier and Type	Field	Description
private static final int	**ITERATIONS**	
private static final int	**KEY_LENGTH**	

Constructor Summary

Constructors

Constructor	Description
AES()	

Method Summary

All Methods Static Methods Concrete Methods

Modifier and Type	Method	Description
static String	**decrypt**(String encrypted, SecretKey key)	We use Base64 as the encoder so we first have to get our bytes decoded back into bytes, and

		we know the structure of the cipher text follows that the first 12 bytes is the IV then follows the cipher text and then the GCM tag which is 128 bits.
static SecretKey	**deriveKey**(String password, byte[] salt)	Uses master password to convert it to a character array, the salt we generated above, we then use factory with the PBKDF2WithHmacSHA256 algorithm to derive the key.
static String	**encrypt**(String plaintext, SecretKey key)	Takes the key and text we need to make cipher text, we use cipher to show how are we creating this cipher text We then use a random IV so that the encryption isn't deterministic.
static byte[]	**generateSalt**()	Salt is used to slow down brute force attacks for the passwords that do get validated and to decrease pattern recognition.

Methods inherited from class Object

clone , equals , finalize , getClass , hashCode , notify , notifyAll , toString , wait , wait , wait

Field Details

KEY_LENGTH

```
private static final int KEY_LENGTH
```

See Also:

Constant Field Values

ITERATIONS

```
private static final int ITERATIONS
```

See Also:

Constant Field Values

Constructor Details

AES

```
public AES()
```

Method Details

generateSalt

```
public static byte[] generateSalt()
```

Salt is used to slow down brute force attacks for the passwords that do get validated and to decrease pattern recognition. Uses a byte array with the size 16 as that is a good standard strength for salt generation. Could also use 8 byte or 32 byte. We use Secure Random because it goes based of hardware RNG and not a set algorithm like Math.random. We then fill the byte array with random bytes and return it as the salt where every 6 bits is a character through Base64

Returns:

a new 16-byte random salt

deriveKey

```
public static SecretKey  deriveKey(String  password,
                                   byte[] salt)
                            throws Exception
```

Uses master password to convert it to a character array, the salt we generated above, we then use factory with the PBKDF2WithHmacSHA256 algorithm to derive the key. We basically hash the password and salt combination the number of iterations and create the key of that result

Parameters:

`password` - the plaintext password to derive a key from

`salt` - the random salt

Returns:

a `SecretKey` suitable for AES encryption/decryption

Throws:

`Exception` - if the key derivation algorithm is unavailable

encrypt

```
public static String encrypt(String plaintext,
                             SecretKey key)
                      throws Exception
```

Takes the key and text we need to make cipher text, we use cipher to show how are we creating this cipher text We then use a random IV so that the encryption isn't deterministic. We then have a 128 tag length used for authentication to make sure data hasn't been tampered with which is what GCM does. We then initialize the encryption using cipher.init and turning our input into bytes. After we then actually encrypt which gives us the authentication tag and cipher text to where we combine the IV with the cipher text and the tag and store it in the database eventually. We then convert it to a string so we can store it in the database as a string and not bytes

Parameters:

`plaintext` - the plaintext string to encrypt

`key` - the AES `SecretKey` to use for encryption

Returns:

a Base64-encoded string containing the IV followed by the ciphertext

Throws:

`Exception` - if encryption fails

decrypt

```
public static String decrypt(String encrypted,
                             SecretKey key)
                      throws Exception
```

We use Base64 as the encoder so we first have to get our bytes decoded back into bytes,

and we know the structure of the cipher text follows that the first 12 bytes is the IV then follows the cipher text and then the GCM tag which is 128 bits. We then take apart the IV by creating a variable 12 bytes long, and then we have the cipher text and GCM tag that can be decrypted by the key now that we know that. We then call the same encryption type as we did in encrypt and initialize it for decrypt mode with the key and tag specification to where we store the decrypted text into a byte array and return it cast as a string so java does its own encoding.

Parameters:

`encrypted` - the Base64-encoded encrypted string IV, Cipher text and GCM tag

`key` - the AES `SecretKey` to use for decryption

Returns:

the original plaintext string

Throws:

`Exception` - if decryption fails

Class DBQueries

java.lang.Object
com.example.passmanager.DBQueries

```
public class DBQueries
extends Object
```

This class is used to have a place where all SQL statements passed to the backend can live, although these are susceptible to SQL injections the next scope will be to change that

Constructor Summary

Constructors

Constructor	Description
`DBQueries()`	Constructs a new DBQueries instance.

Method Summary

All Methods Instance Methods Concrete Methods

Modifier and Type	Method	Description
String	`addAccount(String platform, String username, String password, String salt)`	Returns an INSERT statement to add a new account record.
String	`checkForAccount(String username, String platform)`	Returns a SELECT statement to check whether an account already exists for the given username and platform.
String	`getAccountByUsername(String username, String platform)`	Returns a SELECT statement to retrieve the PASSWORD and RANDOM_SALT for an account matching the given username and platform.

String	**getUsernameByID**(int ID)	Returns a SELECT statement to retrieve the USERNAME for a given account ID.
String	**removeAccount**(String platform, String username)	Returns a DELETE statement to remove an account matching the platform and username.
String	**resultSetToString**(ResultSet rs)	Reads the first row of a result set and returns its contents as a formatted string.
String	**viewAccountInfo**(String searchedFor)	Returns a SELECT statement to find accounts matching a platform, username, or password.

Methods inherited from class Object

clone , equals , finalize , getClass , hashCode , notify , notifyAll , toString , wait , wait , wait

Constructor Details

DBQueries

```
public DBQueries()
```

Constructs a new DBQueries instance.

Method Details

addAccount

```
public String  addAccount(String  platform,
                          String  username,
                          String  password,
                          String  salt)
                  throws Exception
```

Returns an INSERT statement to add a new account record.

Parameters:

`platform` - the platform/service name

`username` - the account username

`password` - the encrypted password

`salt` - the Base64-encoded salt used during encryption

Returns:

a SQL INSERT string for the Accounts table

Throws:

`Exception` - if the query cannot be constructed

removeAccount

```
public String  removeAccount(String  platform,
                             String  username)
                     throws Exception
```

Returns a DELETE statement to remove an account matching the platform and username.

Parameters:

`platform` - the platform name to match

`username` - the username to match

Returns:

a SQL DELETE string for the Accounts table

Throws:

`Exception` - if the query cannot be constructed

viewAccountInfo

```
public String  viewAccountInfo(String  searchedFor)
                       throws Exception
```

Returns a SELECT statement to find accounts matching a platform, username, or password.

Parameters:

`searchedFor` - the value to search across Platform, Username, and Password columns

Returns:

a SQL SELECT string for the Accounts table

Throws:

`Exception` - if the query cannot be constructed

checkForAccount

```
public String  checkForAccount(String  username,
                               String  platform)
                        throws Exception
```

Returns a SELECT statement to check whether an account already exists for the given username and platform.

Parameters:

`username` - the username to look up

`platform` - the platform name to match

Returns:

a SQL SELECT string for the Accounts table

Throws:

`Exception` - if the query cannot be constructed

getAccountByUsername

```
public String  getAccountByUsername(String  username,
                                    String  platform)
                             throws Exception
```

Returns a SELECT statement to retrieve the PASSWORD and RANDOM_SALT for an account matching the given username and platform.

Parameters:

`username` - the username to look up

`platform` - the platform name to match

Returns:

a SQL SELECT string returning PASSWORD and RANDOM_SALT columns

Throws:

`Exception` - if the query cannot be constructed

getUsernameByID

```
public String getUsernameByID(int ID)
                        throws Exception
```

Returns a SELECT statement to retrieve the USERNAME for a given account ID.

Parameters:

`ID` - the account ID to look up

Returns:

a SQL SELECT string returning the USERNAME column

Throws:

`Exception` - if the query cannot be constructed

resultSetToString

```
public String resultSetToString(ResultSet rs)
```

Reads the first row of a result set and returns its contents as a formatted string.

Parameters:

`rs` - the result set to read from

Returns:

a string containing ID, Platform, Username, Password, and Salt from the first row, or an empty string if the result set is empty

Class HelloApplication

java.lang.Object
    javafx.application.Application
        com.example.passmanager.HelloApplication

```
public class HelloApplication
extends javafx.application.Application
```

JavaFX class that extends Application in order to be the entry point for the UI and allows the creation of stages and scenes on top of it

Nested Class Summary

Nested classes/interfaces inherited from class javafx.application.Application

`javafx.application.Application.Parameters`

Field Summary

Fields inherited from class javafx.application.Application

`STYLESHEET_CASPIAN, STYLESHEET_MODENA`

Constructor Summary

Constructors

Constructor	Description
`HelloApplication()`	

Method Summary

All Methods Instance Methods Concrete Methods

Modifier and Type	Method	Description
void	`start`	Called at runtime and loads the

(javafx.stage.Stage stage)	FXML file to be the first screen seen to which we can then interact with.

Methods inherited from class javafx.application.Application

`getHostServices, getParameters, getUserAgentStylesheet, init, launch, launch, notifyPreloader, setUserAgentStylesheet, stop`

Methods inherited from class Object

`clone` , `equals` , `finalize` , `getClass` , `hashCode` , `notify` , `notifyAll` , `toString` , `wait` , `wait` , `wait`

Constructor Details

HelloApplication

```
public HelloApplication()
```

Method Details

start

```
public void start(javafx.stage.Stage stage)
           throws IOException
```

Called at runtime and loads the FXML file to be the first screen seen to which we can then interact with. Also sets the header of the windows name. Instantiates JavaFX

Specified by:

`start` in class `javafx.application.Application`

Parameters:

`stage` - the primary stage provided by the JavaFX runtime

Throws:

`IOException` - if the FXML resource cannot be loaded

Class Launcher

java.lang.Object
com.example.passmanager.Launcher

```
public class Launcher
extends Object
```

The point where the application is loaded from

Constructor Summary

Constructors

Constructor	Description
`Launcher()`	

Method Summary

All Methods Static Methods Concrete Methods

Modifier and Type	Method	Description
`static void`	`main(String [] args)`	Main method — sets up the H2 database server and launches the JavaFX application.

Methods inherited from class Object

`clone` , `equals` , `finalize` , `getClass` , `hashCode` , `notify` , `notifyAll` , `toString` , `wait` , `wait` , `wait`

Constructor Details

Launcher

```
public Launcher()
```

Method Details

main

```
public static void main(String [] args)
                throws SQLException
```

Main method — sets up the H2 database server and launches the JavaFX application. Calls Hello Application to instantiate JavaFX and first stage

Parameters:

`args` - command-line arguments passed through to JavaFX

Throws:

`SQLException` - if the database connection or H2 web server cannot be started

Class MainScreenController

java.lang.Object
com.example.passmanager.MainScreenController

```
public class MainScreenController
extends Object
```

This is used as the controller for the main screen of the application. Used for navigation and interacting with the observable list

Field Summary

Fields

Modifier and Type	Field	Description
private javafx.collections.ObservableList<AccountInstance>	accounts	
private javafx.scene.control.ListView<AccountInstance>	accountsList	

Constructor Summary

Constructors

Constructor	Description
MainScreenController()	

Method Summary

All Methods Instance Methods Concrete Methods

Modifier and Type	Method	Description
private void	deleteAccount(AccountInstance account)	This calls a query from DBQueries to get the account associated with the username

		and platform given and removes it from the database
void	**initialize**()	Called automatically by JavaFX after the FXML scene is loaded.
private void	**loadAccountsFromDB**()	This queries the database to get all the accounts available and when the H2 database returns the result set we format into account objects that we add to an observable list until the result set has no more returns.
private void	**onAddAccountClick**()	When the Add Account button is clicked we create a pop-up asking for the necessary information, as long as the information passes the validation format from AddAccountController the account gets created and the listview is updated by calling all the accounts from the database again.
private void	**showAccountPopup** (AccountInstance account)	This creates the pop-up window for when an account is selected in the list view.

Methods inherited from class Object

clone , equals , finalize , getClass , hashCode , notify , notifyAll , toString , wait , wait , wait

Field Details

accountsList

```
private javafx.scene.control.ListView<AccountInstance> accountsList
```

accounts

```
private javafx.collections.ObservableList<AccountInstance> accounts
```

Constructor Details

MainScreenController

```
public MainScreenController()
```

Method Details

initialize

```
public void initialize()
```

Called automatically by JavaFX after the FXML scene is loaded. We create a list view here that we can interact with which is what initialize is important. We basically are saying hey lets get the accounts from the database, create a list of them and then display them ina list view while we create a click listener that displays a pop-up for that specific account if it is clicked by calling showAccountPopup.

loadAccountsFromDB

```
private void loadAccountsFromDB()
```

This queries the database to get all the accounts available and when the H2 database returns the result set we format into account objects that we add to an observable list until the result set has no more returns. We then set the observable list into accountsList that then creates cells for each AccountInstance and highlights the account where the primary key ID is 1

showAccountPopup

```
private void showAccountPopup(AccountInstance account)
```

This creates the pop-up window for when an account is selected in the list view. You have to actions to close the window or delete the account. Session creates a static

instance of the password so all controllers can access the main password. We then check if the account is the logged in account and if it is we do not display a delete button. We then wait for user interaction and if the click delete account the account is deleted

deleteAccount

```
private void deleteAccount(AccountInstance account)
```

This calls a query from DBQueries to get the account associated with the username and platform given and removes it from the database

onAddAccountClick

```
private void onAddAccountClick()
```

When the Add Account button is clicked we create a pop-up asking for the necessary information, as long as the information passes the validation format from AddAccountController the account gets created and the listview is updated by calling all the accounts from the database again.

Class ModelController

java.lang.Object
com.example.passmanager.ModelController

```
public class ModelController
extends Object
```

This controller is used to handle account creation, logging in, and successful login attempts vs unsuccessful

Field Summary

Fields

Modifier and Type	Field	Description
private javafx.scene.control.ListView<String>	**accountsList**	
private javafx.scene.control.ListView<String>	**alertsList**	
private javafx.scene.control.PasswordField	**confirmPasswordField**	
private javafx.scene.control.Label	**createStatusLabel**	
(package private) AccountInstance	**currentAccount**	
private javafx.scene.control.DatePicker	**dobPicker**	
private javafx.scene.control.TextField	**firstNameField**	
private javafx.scene.control.TextField	**lastNameField**	
private javafx.scene.control.PasswordField	**newPasswordField**	
private javafx.scene.control.TextField	**newUsernameField**	
private javafx.scene.control.PasswordField	**passwordField**	
(package private) DBQuerier	**query**	

(package private) DbQueries	**query**
(package private) SignUp	**signUp**
private javafx.scene.control.Label	**statusLabel**
private javafx.scene.control.TextField	**usernameField**

Constructor Summary

Constructors

Constructor	Description
ModelController()	

Method Summary

All Methods Instance Methods Concrete Methods

Modifier and Type	Method	Description
protected void	**onBackClick** (javafx.event.ActionEvent event	Navigates back to the welcome screen.
void	**onCreateAccountClick** (javafx.event.ActionEvent event	Navigates to the create-account form view.
void	**onCreateAccountSubmit** (javafx.event.ActionEvent event	Handles submission of the create-account form.
void	**onLoginClick** (javafx.event.ActionEvent event	Navigates to the login view.
protected void	**onLoginSubmit** (javafx.event.ActionEvent event	Handles login form submission.

Methods inherited from class Object

clone , equals , finalize , getClass , hashCode , notify , notifyAll ,

toString , wait , wait , wait

Field Details

currentAccount

AccountInstance currentAccount

signUp

SignUp signUp

query

DBQueries query

firstNameField

private javafx.scene.control.TextField firstNameField

lastNameField

private javafx.scene.control.TextField lastNameField

newUsernameField

private javafx.scene.control.TextField newUsernameField

newPasswordField

private javafx.scene.control.PasswordField newPasswordField

confirmPasswordField

private javafx.scene.control.PasswordField confirmPasswordField

dobPicker

```
private javafx.scene.control.DatePicker dobPicker
```

createStatusLabel

```
private javafx.scene.control.Label createStatusLabel
```

usernameField

```
private javafx.scene.control.TextField usernameField
```

passwordField

```
private javafx.scene.control.PasswordField passwordField
```

statusLabel

```
private javafx.scene.control.Label statusLabel
```

alertsList

```
private javafx.scene.control.ListView<String > alertsList
```

accountsList

```
private javafx.scene.control.ListView<String > accountsList
```

Constructor Details

ModelController

```
public ModelController()
```

Method Details

onLoginClick

```
public void onLoginClick(javafx.event.ActionEvent event)
                  throws IOException
```

Navigates to the login view.

Parameters:

`event` - the action event from the Login button

Throws:

`IOException` - if the login-view FXML cannot be loaded

onCreateAccountClick

```
public void onCreateAccountClick(javafx.event.ActionEvent event)
                          throws IOException
```

Navigates to the create-account form view.

Parameters:

`event` - the action event from the Create Account button

Throws:

`IOException` - if the create-account FXML cannot be loaded

onCreateAccountSubmit

```
public void onCreateAccountSubmit(javafx.event.ActionEvent event)
                           throws Exception
```

Handles submission of the create-account form.

Validates that all fields are filled and passwords match, derives an AES key, encrypts the password, checks for duplicate accounts, persists the new account to the database, and redirects to the login view on success.

Parameters:

`event` - the action event from the Submit button

Throws:

`Exception` - if encryption, key derivation, or database access fails

onLoginSubmit

```
protected void onLoginSubmit(javafx.event.ActionEvent event)
                      throws Exception
```

Handles login form submission. Looks up the stored encrypted password and salt for the username, re-creates the AES key from the entered password, decrypts the stored password, and compares it. On success, stores the key in the Session class method setKey and navigates to the main screen.

Parameters:

`event` - the action event from the Login button

Throws:

`Exception` - if decryption, key derivation, or database access fails

onBackClick

```
protected void onBackClick(javafx.event.ActionEvent event)
                    throws IOException
```

Navigates back to the welcome screen.

Parameters:

`event` - the action event from the Back button

Throws:

`IOException` - if the first-scene FXML cannot be loaded

Class Session

java.lang.Object
com.example.passmanager.Session

```
public class Session
extends Object
```

Holds the AES key for the entire app for the duration of a user session derived from initial log in The controllers can access the key without passing it through every method call.

Field Summary

Fields

Modifier and Type	Field	Description
private static SecretKey	**key**	

Constructor Summary

Constructors

Constructor	Description
Session()	

Method Summary

All Methods Static Methods Concrete Methods

Modifier and Type	Method	Description
static SecretKey	**getKey**()	Returns the session's AES key.
static void	**setKey**(SecretKey k)	Stores the session key after a successful login.

Methods inherited from class Object

`clone` , `equals` , `finalize` , `getClass` , `hashCode` , `notify` , `notifyAll` , `toString` , `wait` , `wait` , `wait`

Field Details

key

```
private static SecretKey  key
```

Constructor Details

Session

```
public Session()
```

Method Details

setKey

```
public static void setKey(SecretKey  k)
```

Stores the session key after a successful login.

Parameters:

k - the derived `SecretKey` to store

getKey

```
public static SecretKey  getKey()
```

Returns the session's AES key.

Returns:

the key set at login, or it's null if it has not been set

Class SignUp

java.lang.Object
com.example.passmanager.AccountInstance
com.example.passmanager.SignUp

```
public class SignUp
extends AccountInstance
```

Represents the primary user account created during sign-up Extends AccountInstance with personal details (first name, last name, date of birth) and the salt used to derive the user's AES key. The platform for this account is always MM PassManager and the ID is always 1

Field Summary

Fields

Modifier and Type	Field	Description
private LocalDate	**dateOfBirth**	
private String	**fname**	
private String	**lname**	
private String	**salt**	

Constructor Summary

Constructors

Constructor	Description
SignUp(String password, String platform, String username, String fname, String lname, LocalDate dateOfBirth, String salt)	Constructs a new SignUp account with full user profile information.

Method Summary

All Methods Instance Methods Concrete Methods

Modifier and Type	Method	Description
LocalDate	**getDateOfBirth**()	Returns the user's date of birth.
String	**getFname**()	Returns the user's first name.
String	**getLname**()	Returns the user's last name.
void	**setDateOfBirth**(LocalDate dateOfBirth)	Sets the user's date of birth.
void	**setFname**(String fname)	Sets the user's first name if the string is not empty.
void	**setLname**(String lname)	Sets the user's last name if the string is not empty.

Methods inherited from class AccountInstance

getId, getPassword, getPlatform, getUsername, setPassword, setPlatform, setUsername

Methods inherited from class Object

clone , equals , finalize , getClass , hashCode , notify , notifyAll , toString , wait , wait , wait

Field Details

fname

private String fname

lname

private String lname

dateOfBirth

```
private LocalDate  dateOfBirth
```

salt

```
private String  salt
```

Constructor Details

SignUp

```
public SignUp(String  password,
              String  platform,
              String  username,
              String  fname,
              String  lname,
              LocalDate  dateOfBirth,
              String  salt)
```

Constructs a new SignUp account with full user profile information.

Parameters:

`password` - the encrypted password for this account

`platform` - the platform name MM PassManager

`username` - the username

`fname` - the user's first name

`lname` - the user's last name

`dateOfBirth` - the user's date of birth

`salt` - the encoded salt

Method Details

getFname

```
public String  getFname()
```

Returns the user's first name.

Returns:

the first name

setFname

```
public void setFname(String  fname)
```

Sets the user's first name if the string is not empty.

Parameters:

`fname` - the new first name; ignored if blank

getLname

```
public String  getLname()
```

Returns the user's last name.

Returns:

the last name

setLname

```
public void setLname(String  lname)
```

Sets the user's last name if the string is not empty.

Parameters:

`lname` - the new last name; ignored if blank

getDateOfBirth

```
public LocalDate  getDateOfBirth()
```

Returns the user's date of birth.

Returns:

the date of birth as a LocalDate object

setDateOfBirth

```
public void setDateOfBirth(LocalDate  dateOfBirth)
```

Sets the user's date of birth.

Parameters:

`dateOfBirth` - the new date of birth

Module com.example.passmanager

module com.example.passmanager

Packages

All Packages Exports Opens

Package	Exported To Modules	Opened To Modules	Description
com.example.passmanager	All Modules	javafx.fxml	

Indirect Exports

From	Packages
com.h2database	org.h2 org.h2.api org.h2.bnf org.h2.bnf.context org.h2.command org.h2.command.ddl org.h2.command.dml org.h2.command.query org.h2.compress org.h2.constraint org.h2.engine org.h2.expression org.h2.expression.aggregate org.h2.expression.analysis org.h2.expression.condition org.h2.expression.function org.h2.expression.function.table org.h2.fulltext org.h2.index org.h2.jdbc org.h2.jdbc.meta org.h2.jdbcx org.h2.jmx org.h2.message org.h2.mode org.h2.mvstore org.h2.mvstore.cache org.h2.mvstore.db org.h2.mvstore.rtree org.h2.mvstore.tx org.h2.mvstore.type org.h2.result org.h2.schema org.h2.security org.h2.security.auth org.h2.security.auth.impl org.h2.server org.h2.server.pg org.h2.server.web org.h2.store org.h2.store.fs org.h2.store.fs.async org.h2.store.fs.disk org.h2.store.fs.encrypt org.h2.store.fs.mem org.h2.store.fs.niomapped org.h2.store.fs.niomem org.h2.store.fs.rec org.h2.store.fs.retry org.h2.store.fs.split org.h2.store.fs.zip org.h2.table org.h2.tools org.h2.util org.h2.util.geometry org.h2.util.json org.h2.value org.h2.value.lob
java.base	com.sun.crypto.provider com.sun.security.ntlm java.io java.lang java.lang.annotation java.lang.classfile java.lang.classfile.attribute java.lang.classfile.constantpool java.lang.classfile.instruction java.lang.constant java.lang.foreign java.lang.invoke java.lang.module java.lang.ref java.lang.reflect java.lang.runtime

	java.math java.net java.net.spi java.nio java.nio.channels java.nio.channels.spi java.nio.charset java.nio.charset.spi java.nio.file java.nio.file.attribute java.nio.file.spi java.security java.security.cert java.security.interfaces java.security.spec java.text java.text.spi java.time java.time.chrono java.time.format java.time.temporal java.time.zone java.util java.util.concurrent java.util.concurrent.atomic java.util.concurrent.locks java.util.function java.util.jar java.util.random java.util.regex java.util.spi java.util.stream java.util.zip javax.crypto javax.crypto.interfaces javax.crypto.spec javax.net javax.net.ssl javax.security.auth javax.security.auth.callback javax.security.auth.login javax.security.auth.spi javax.security.auth.x500 javax.security.cert jdk.internal jdk.internal.access jdk.internal.classfile.components jdk.internal.event jdk.internal.foreign jdk.internal.io jdk.internal.javac jdk.internal.jimage jdk.internal.jimage.decompressor jdk.internal.jmod jdk.internal.loader jdk.internal.logger jdk.internal.misc jdk.internal.module jdk.internal.org.xml.sax jdk.internal.org.xml.sax.helpers jdk.internal.perf jdk.internal.platform jdk.internal.ref jdk.internal.reflect jdk.internal.util jdk.internal.util.xml jdk.internal.util.xml.impl jdk.internal.vm jdk.internal.vm.annotation jdk.internal.vm.vector sun.invoke.util sun.net sun.net.dns sun.net.ext sun.net.util sun.net.www sun.net.www.protocol.http sun.nio.ch sun.nio.cs sun.nio.fs sun.reflect.annotation sun.reflect.generics.reflectiveObjects sun.reflect.misc sun.security.internal.interfaces sun.security.internal.spec sun.security.jca sun.security.pkcs sun.security.provider sun.security.provider.certpath sun.security.rsa sun.security.timestamp sun.security.tools sun.security.util sun.security.validator sun.security.x509 sun.util.cldr sun.util.locale.provider sun.util.logging sun.util.resources
java.desktop	java.applet java.awt java.awt.color java.awt.desktop java.awt.dnd java.awt.dnd.peer java.awt.event java.awt.font java.awt.geom java.awt.im java.awt.im.spi java.awt.image java.awt.image.renderable java.awt.print java.beans java.beans.beancontext javax.accessibility javax.imageio javax.imageio.event javax.imageio.metadata javax.imageio.plugins.bmp javax.imageio.plugins.jpeg javax.imageio.plugins.tiff javax.imageio.spi javax.imageio.stream javax.print javax.print.attribute javax.print.attribute.standard javax.print.event javax.sound javax.sound.midi javax.sound.midi.spi javax.sound.sampled javax.sound.sampled.spi javax.swing

	javax.swing.border javax.swing.colorchooser javax.swing.event javax.swing.filechooser javax.swing.plaf javax.swing.plaf.basic javax.swing.plaf.metal javax.swing.plaf.multi javax.swing.plaf.nimbus javax.swing.plaf.synth javax.swing.table javax.swing.text javax.swing.text.html javax.swing.text.html.parser javax.swing.text.rtf javax.swing.tree javax.swing.undo sun.awt sun.awt.dnd sun.swing
java.sql	java.sql javax.sql
javafx.controls	com.sun.javafx.scene.control com.sun.javafx.scene.control.behavior com.sun.javafx.scene.control.inputmap com.sun.javafx.scene.control.skin javafx.scene.chart javafx.scene.control javafx.scene.control.cell javafx.scene.control.skin
javafx.fxml	javafx.fxml

Indirect Opens

From	Packages
java.desktop	javax.swing.plaf.basic

Modules

Requires

Modifier	Module	Description
	com.h2database	
	java.base	
	java.desktop	
	java.sql	
	javafx.controls	
	javafx.fxml	